



PAL INFORMATICA

Besturingssystemen - Sessie 2

Yarne Ramakers, Jonas Couwberghs – 30/04/2026

Inhoud



1. Virtual Memory
2. Page Tables
3. Page Faults
4. File systems
5. Vragen over de Labos
6. Voorbeeld examenvraag



VIRTUAL MEMORY

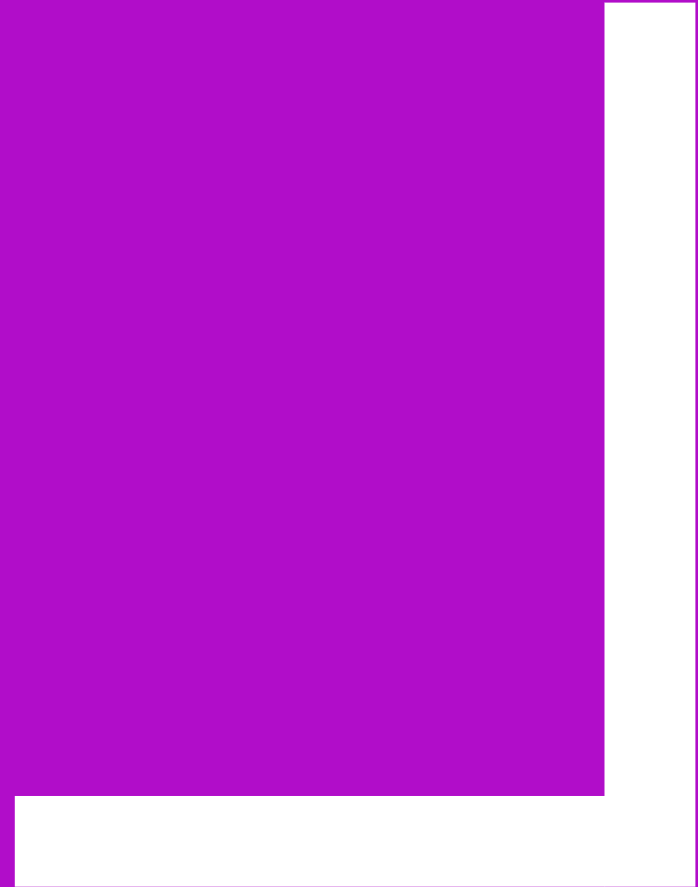
Virtual Memory



- ▶ Een abstractie die elk proces de indruk geeft dat het een **eigen, continu** blok geheugen bezit
- ▶ Processen kunnen elkaars data niet corrupt maken
- ▶ Kernel beheert de fysieke frames
- ▶ **MMU** vertaalt elke virtuele adresoproep naar fysiek RAM



PAGE TABLES



Waarom?



- ▶ Bepaalt wie mag lezen, schrijven of uitvoeren via permissiebits in PTE
- ▶ Alleen nodige geheugen gealloceerd (demand paging)
- ▶ Geheugen hoeft niet fysiek aaneengesloten te zijn (virtueel lijkt dit wel zo)

RISC-V Sv39



xv6 RISC-V gebruikt de Sv39 standaard

- ▶ 39 bits gebruikt van een adres
- ▶ 27 bits voor VPN, 12 voor offset
- ▶ 9 bits per index voor elk niveau (3 niveaus)
- ▶ offset is 12 bits (4096 bytes per page)
- ▶ niet gebruikte MSB worden gesign-extend
 - zodat oude software niet breekt als je naar Sv48 of Sv57 gaat
 - xv6 sign extend niet (meer hierover verder)

oude xv6 had maar 2 niveaus

Note: OSTEP gebruikt 4 niveau (komt overeen met de Sv48 standaard) page tables (meest gebruikt in praktijk)

Soorten tabellen

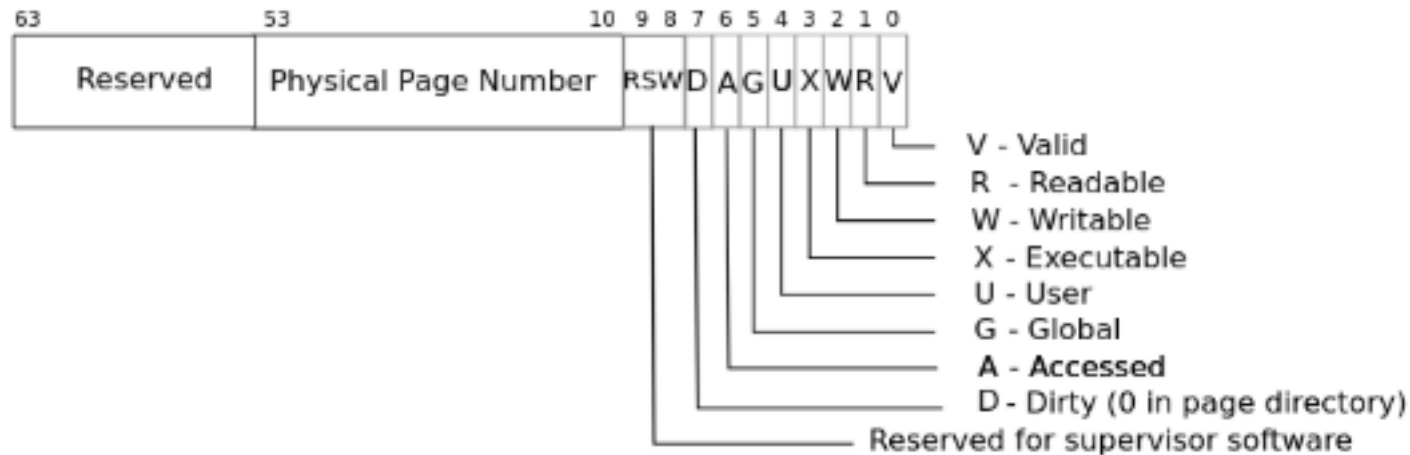


- ▶ lineaire tabellen
 - één enorme array
 - bij 64 bit adressen petabytes aan RAM
 - ookal gebruiken ze maar 1KB
- ▶ multi level
 - boomstructuur
 - als heel adresbereik niet gebruikt wordt, hoeven onderliggende tabellen niet te bestaan

Page Table Entry (PTE)



- ▶ 10 reserved bits door de RISC-V specificatie
- ▶ 44 bits PPN: fysieke adres van de page frame in RAM
- ▶ 2 bits RSW: kernel mag zelf kiezen hoe ze deze gebruikt
- ▶ 8 bits flags



Flags



- ▶ Dirty: hardware zet dit op 1 als naar page geschreven wordt
- ▶ Accessed: hardware zet op 1 als gelezen/geschreven wordt
 - kernel gebruikt dit om te zien welke gebruikt worden
- ▶ Global: mapping is geldig in alle virtual address spaces
 - zodat CPU de TLB niet hoeft te flushen bij een context switch
- ▶ User: als 0 kan enkel de Kernel aan deze entry
- ▶ Execute: als 1 mag de CPU code uitvoeren in deze page
- ▶ Write: als 1 mag een programma de data aanpassen
- ▶ Read: als 1 mag een programma lezen
- ▶ Valid: als 0 doet de CPU alsof de PTE niet bestaat => pagefault

Note: Als R,W en X op 0 is en $V = 1$ is het een pointer naar de volgende page table

xv6 functies



- ▶ `kalloc(void)` geeft vrije fysieke page terug
- ▶ `memset(void*, int, uint)` zet een stuk geheugen op een bepaalde waarde (wissen)
- ▶ `mappages(pagetable_t, uint64, uint64, uint64, int)` voegt PTE's toe aan de PT
- ▶ `uvmcopy(pagetable_t, pagetable_t, uint64)` kopieer geheugen van parent naar kind
 - zowel PT als fysieke geheugen
- ▶ `uvmunmap(pagetable_t, uint64, uint64, int)` remove n pages van de mapping
- ▶ `uvmfree(pagetable_t, uint64)` free een process's PT en het fysieke geheugen
- ▶ `uvmcreate(void)` create een lege user page table

Note: `pagetable_t` is gewoon een `uint64`

Walk en walkaddr



Functies als `uvmunmap` gebruiken `walk()` en `walkaddr()` om dingen met pages te doen

- ▶ `walk(pagetable_t, uint64, int)` retournt adres van PTE in de pagetable
 - simuleert de walk die de MMU doet
 - retournt een pointer naar de leaf PTE
 - kan de nodige pagetables ook alloceren
- ▶ `walkaddr(pagetable_t, uint64)` retournt het fysieke adres van de PTE
 - checkt of de page gemapt is in user space
- ▶ `freewalk(pagetable_t)` freet alle pages

Macro's en constants



- ▶ `#define PGSIZE 4096` page size
- ▶ `#define PGSHIFT 12` offset in een page
- ▶ `#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))`
 - xv6 devs limiten adressen tot 38 bits zodat bit 39 altijd 0 is
- ▶ `#define PGROUNDUP(sz) (((sz)+PGSIZE-1) & ~(PGSIZE-1))`
- ▶ `#define PGROUNDDOWN(a) (((a)) & ~(PGSIZE-1))`
- ▶ `#define PA2PTE(pa) (((uint64)pa) >> 12) << 10)`
- ▶ `#define PTE2PA(pte) (((pte) >> 10) << 12)`
- ▶ `#define PX(level, va) (((uint64)(va)) >> PXSHIFT(level)) & PXMASK)`

Virtual Dynamically-linked Shared Object



- ▶ Vermeiden dat process constant syscalls doet voor bv de tijd op te vragen
- ▶ Kernel maakt een page in het fysieke ram en houdt hier bv `ticks` bij
 - kernel houdt deze up to date
- ▶ Kernel mapt dit in de user space van elk proces
 - `PTE_R` | `PTE_U`, process mag niet schrijven!
- ▶ Proces kan nu uit zijn eigen geheugen lezen ipv trappen naar de kernel

Zelf vDSO pages maken



1. Plaats variabele in een eigen sectie, vb in `kernel/trap.c` variabele afzonderen

```
uint ticks __attribute__((section(".vdso")));
```

2. Pas linker aan in `kernel/linker.ld`

- zorg dat de sectie exact één page (4096 bytes) groot is, plaats dit net boven
`PROVIDE(end = .);`
 - `end` markeert het einde van de kernel in fysieke RAM

```
.vdso : {  
    . = ALIGN(0x1000); /* Uitlijnen op page boundary */  
    _vdso_start = .;   /* Maak pointer naar startadres */  
    *(.vdso);          /* Plaats variabelen hier */  
    . = ALIGN(0x1000); /* Lijn einde uit */  
    ASSERT(. - _vdso_start == 0x1000, "vdso groter dan 1 page");  
}
```



3. Mappen in elk user proces kernel/proc.c

- haal de startpointer op via `extern char _vdso_start[]` en voeg toe aan `proc_pagetable()`

```
// Offset is meestal net onder de trapframe
uint64 offset = TRAPFRAME - PGSIZE;
if(mappages(pagetable, offset, PGSIZE, (uint64)_vdso_start, PTE_R | PTE_U) < 0){
    // Mislukt? Unmap dan de eerder gemapte pages en free de pagetable!
}
```

Note: Zorg dat de permissies enkel `PTE_R | PTE_U` zijn. Geef nooit `PTE_W` (Write) permissie mee, anders kan de user kernel-data zoals de systeemtijd overschrijven!



4. Als proces stopt moet mapping weg via `proc_freepagetable()`

```
// Laatste argument is 0: unmap de PTE, maar free de fysieke kernel page NIET!  
uvmunmap(pagetable, offset, 1, 0);
```



PAGE FAULTS

Wat is een Page Fault?



- ▶ Een hardware **trap** die optreedt als een programma geheugen opvraagt, maar de MMU het niet kan vertalen of niet toelaat
- ▶ Bij een trap vult de CPU twee registers in
 - `scause` (supervisor cause) die vertelt waarom de trap gebeurde
 - ▶ code 12 is instruction page fault
 - ▶ code 13 is load page fault (lezen)
 - ▶ code 15 is store page fault (schrijven)
 - `stval` (supervisor trap value) bevat het **virtuele adres** dat de fout veroorzaakte

Note: Een Page Fault is vaak géén error, maar een iets dat de OS gebruikt om slim met geheugen om te gaan!

Scenario's



1. Segmentation fault

- ▶ programma leest geheugen dat niet gemapt is ($\text{PTE}_V=0$) of schrijft naar read-only geheugen ($\text{PTE}_W=0$)
- ▶ kernel killt proces
- ▶ kernel zorgt altijd dat 0×0 niet gemapt is zodat je null pointer exceptions krijgt

2. Demand paging / lazy allocation

- ▶ programma vraagt om RAM via `sbrk` maar OS is lui en past enkel de process size $p \rightarrow sz$ aan zonder pages te mappen
- ▶ wanneer process page aanraakt krijg je een fault
 - kernel ziet dat adres binnen $p \rightarrow sz$ ligt en allocceert nu pas een page met `kalloc()`, mapt deze met `mappages()` en laat CPU opnieuw proberen



3. Swapping

- ▶ page bestaat, maar RAM was vol, dus OS heeft page tijdelijk harde schijf (swap space) geschreven
- ▶ OS pauzeert proces, leest data van disk naar RAM, update PTE en hervat
 - standaard xv6 doet geen swapping, check OSTEP boek voor meer info

CoW, Moo



Copy on write is een klassieke optimalisatie voor `fork()`

- ▶ `fork()` kopieert het fysieke geheugen van parent naar child
- ▶ via copy-on-write
 - `fork()` kopieert enkel de page tables, beide processen wijzen naar zelfde fysieke page in RAM
 - kernel verwijdert schrijf permissies (`PTE_W=0`)
 - zodra één van de processes iets wil aanpassen throwt de CPU een store page fault
 - kernel ziet dit en kopieert dan pas die specifieke fysieke page

Voordeel: Als child na `fork()` meteen `exec()` doet heb je nooit onnodig geheugen gekopieerd

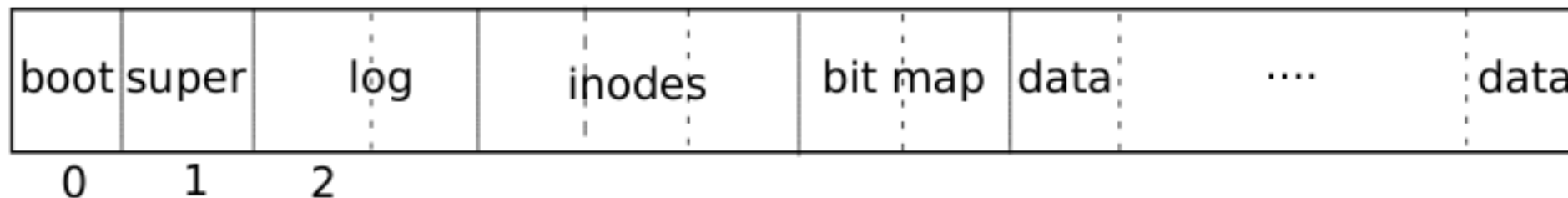


FILE SYSTEMS

Disk Layout

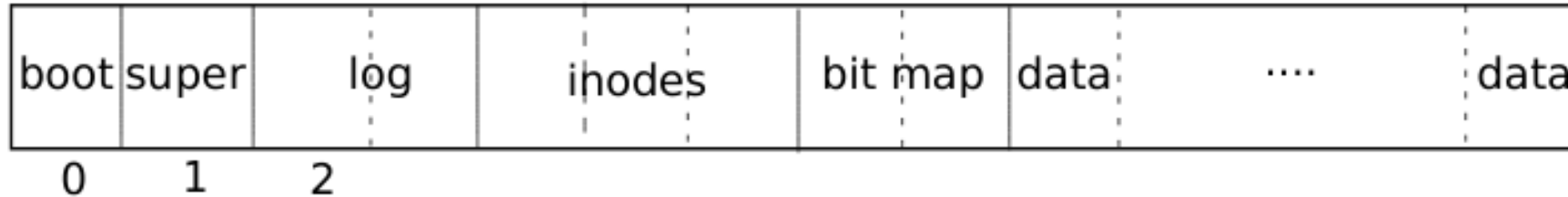


De disk in xv6 is opgedeeld in verschillende blokken van elk 1024 bytes (BSIZE). Het soort data in elk blok is bepaald door de disklayout:



Disk Layout

Verschillende blocks



- ▶ **boot:** Bevat de boot code, standaard het eerste blok (blok 0)
- ▶ **super:** Bevat meta-informatie over het filesystem
- ▶ **log:** Het log dient als een tijdelijk “kladblok” dat alle wijzigingen opslaat voordat ze definitief naar de schijf worden geschreven, zodat het bestandssysteem consistent blijft bij een crash.
- ▶ **inodes:** Bevat alle kenmerken (type, grootte, locatie) van een bestanden.
- ▶ **bitmap:** Houdt bij welke blokken bezet zijn.
- ▶ **data:** De eigenlijke inhoud van de bestanden.

Disk Layout

Vragen over inodes



1. What is the maximum number of blocks per file in xv6? What is the maximum file size?

Disk Layout

Vragen over inodes



1. What is the maximum number of blocks per file in xv6? What is the maximum file size?

Als we kijken naar de constanten gedefinieerd boven de `dinode` struct in `kernel/fs.c` zien we dat een bestand 12 directe blokken bevat, daarnaast bevat het ook een indirect blok nummer die wijst naar een blok dat nog `BSIZE / sizeof(uint) = 1024 / 4 = 256` blokken nummers kan bevatten. Een bestand kan dus in totaal 268 blokken bevatten.

Disk Layout

Vragen over inodes



2. Suppose you have a directory with exactly 1 allocated data block in the inode. What is the maximum number of directory entries this directory can contain?

Disk Layout

Vragen over inodes



2. Suppose you have a directory with exactly 1 allocated data block in the inode. What is the maximum number of directory entries this directory can contain?

Een `dirent` struct is 16 bytes groot, (2 bytes voor `ushort inum` en 14 bytes voor `char name[DIRSIZ]`), in 1 block passen dus $1024 / 16 = 64$ entries.

Disk Layout

Vragen over inodes



3. What is the effective maximum number of entries a directory in xv6 can contain?

Disk Layout

Vragen over inodes



3. What is the effective maximum number of entries a directory in xv6 can contain?

Zoals we in vraag 1 zagen kan een bestand maximaal 268 blokken opnemen.

Omdat we dus per blok van een directory 64 entries kunnen opnemen, kan een directory dus $268 * 64 = 17152$ entries bevatten.

Access Permissions



Een bestand heeft bepaalde permissies nodig:

- ▶ **Read** r : Het bestand kan door een user gelezen worden
- ▶ **Write** w : Het bestand kan door een user aangepast worden
- ▶ **Execute** x : Het bestand kan worden uitgevoerd (niet van toepassing in Lab 6)

De permissies van een bestand worden bijgehouden in de bits van een field `mode`:

- ▶ $0b001 = 1 \rightarrow$ Readable
- ▶ $0b010 = 2 \rightarrow$ Writeable
- ▶ $0b100 = 4 \rightarrow$ Executable

Of een combinatie van de 3

Access Permissions

Bitwise operaties en bitmasks



Om een combinatie van de 3 permissies makkelijk te managen worden bitwise operaties met bitmasks gebruikt.

De waarden uit de vorige slides kunnen gezien worden als masks voor hun specifieke permissie.

- ▶ `M_READ = 0b001`
- ▶ `M_WRITE = 0b010`
- ▶ `M_EXEC = 0b100`

Een permissie enablen met bitwise OR:

```
mode = mode | mask; // 0b001 | 0b010 = 0b011
```

Access Permissions

Bitwise operaties en bitmasks - Vervolg



Een permissie disablen met bitwise NOT en bitwise AND:

```
mode = mode & ~mask; // 0b011 & ~0b001 = 0b011 & 0b110 = 0b010
```

Nakijken of een permissie bestaat met bitwise AND:

```
if (mode & mask == mask) { // 0b011 & 0b010 = 0b010 ⇒ 0b010 = 0b010
    ...
}
```

Je kan de voorgaande operaties ook voor meerdere permissies tegelijk doen door de bitmasks te combineren met een bitwise OR:

```
m_read_and_write = M_READ | M_WRITE; // 0b001 | 0b010 = 0b011
```



VRAGEN OVER DE LABOS

Vragen



Heeft iemand vragen over de voorbije 4 labos?



VOORBEELD EXAMENVRAAG

Voorbeeld Examenvraag



In deze grotere oefening moet je de referentie-implementatie voor de *copy-on-write null-page* oefening van Lab 6 uitbreiden met ondersteuning voor **page deduplicatie**. Dit is een optimalisatietechniek die dynamisch identieke fysieke pagina's binnen de heap van een proces identificeert en deze samenvoegt tot één enkele read-only pagina in het fysieke geheugen.

Template branch: <https://gitlab.kuleuven.be/distrinet/education/operating-systems/kul-xv6/-/tree/template25-dedup>

Voorbeeld Examenvraag

Code skeleton



In deze oefening zal je een nieuwe system call implementeren die volgende stappen uitvoert:

1. Loop door de paginatabellen om alle fysieke geheugenpagina's in het heapgebied van het huidige proces te vinden.
2. Lees en vergelijk de inhoud van alle heap-pagina's om gedupliceerde identifieke fysieke pagina's te ontdekken.
3. Remap de gedupliceerde pagina's naar één enkele read-only fysieke pagina. Indien het proces later deze gedupliceerde pagina probeert te overschrijven, passen we *copy-on-write* semantiek toe, waarbij de kernel transparant de mappings voor het faulting adres aanpast naar een nieuwe read-write kopie van de fysieke pagina.

Voorbeeld Examenvraag

Code skeleton - Vervolg



We hebben al een compleet code-skelet voor de eerste twee stappen gegeven in `kernel/sysdedup.c`. Je hoeft dit bestand niet meet aan te passen; implementeer alleen stap 3 in `kernel/vm.c`.

Voorbeeld Examenvraag

Opgave - Stap 1



Ga stapsgewijs te werk als volgt:

1. Voeg een nieuwe system call toe, die in user space aangeroepen kan worden als `void dedup(void)` en in de xv6 kernel de bestaande functie `uint64 sys_dedup(void)` aanroept.

Voorbeeld Examenvraag

Opgave - Stap 2



2. Implementeer de gegeven functie `uvmdedup(pagetable, va_orig, va_dup)` in `kernel/vm.c` zodat zowel het originele virtuele adres `va_orig` als het duplicaat virtuele adres `va_dup` naar dezelfde copy-on-write fysieke pagina verwijzen, terwijl de fysieke pagina van `va_dup` wordt vrijgegeven. Tip: Zie ook de documentatie in de C code boven de functie. Je mag aannemen dat `va_orig` en `va_dup` valid adressen zijn die altijd correct gemapt zijn in de gegeven page table.
- *Nu zou de basisfunctionaliteit voor het dedupliceren van heap-pagina's moeten werken, zonder ondersteuning voor het vrijgeven en schrijven van gededuplicateerde pagina's.*

Voorbeeld Examenvraag

Opgave - Stap 3



3. Implementeer nu de gegeven functie `uvmcount(pagetable, pa)` in `kernel/vm.c` om het aantal pagina's in een virtuele adresruimte te tellen die naar een specifiek fysiek adres `pa` verwijzen. ***Je mag deze zoekactie beperken tot de eerste 100 pagina's van de adresruimte, dat wil zeggen de virtuele adressen van 0 tot $100 \cdot 4096$.***
- ▶ Nu zou ook het terug vrijgeven van gededuplicateerde pagina's correct moeten werken.

Voorbeeld Examenvraag

Opgave - Stap 4



4. Implementeer ten slotte *copy-on-write* ondersteuning voor het overschrijven van eerder gededuplicateerde pagina's. Breid hiervoor de functie `pagefault(va)` in `kernel/vm.c` uit, zodat bij write page faults in het heapgebied de volgende stappen worden uitgevoerd:
- (a) Zorg ervoor dat je code blijft functioneren voor de null page. Controleer of het faulting virtuele adres momenteel naar de fysieke null-pagina mapt. Als dat het geval is, moet je altijd dupliceren, dus ga direct naar stap 4d.
 - (b) Gebruik `uvmcount` om te controleren of er nog andere heappagina's zijn gemapt naar het faulting virtueel adres.
 - (c) Als er geen andere heappagina's zijn gemapt naar het faulting virtueel adres, heb je geen nieuwe fysieke pagina nodig en mag je de bestaande mapping voor `va` rechtstreeks aanpassen.



- (d) Als er andere heappagina's zijn gemapt naar het faulting virtueel adres, moet je een nieuwe fysieke pagina aanmaken, de inhoud van de originele read-only pagina kopiëren en de nieuw gealloceerde fysieke pagina als read-write mappen voor va.

Voorbeeld Examenvraag

Opgave - Stap 5



Valideer je oplossing met `make test-dedup`.